MODULE 2

Microarchitecture

- Introduction;

Performance analysis;

Single-Cycle Processor - Single Cycle Data_path,

Single Cycle Control;

Pipelined Processor - Pipelined Data Path,

Pipelined Control: Hazards, Solving Data/Control Hazards,

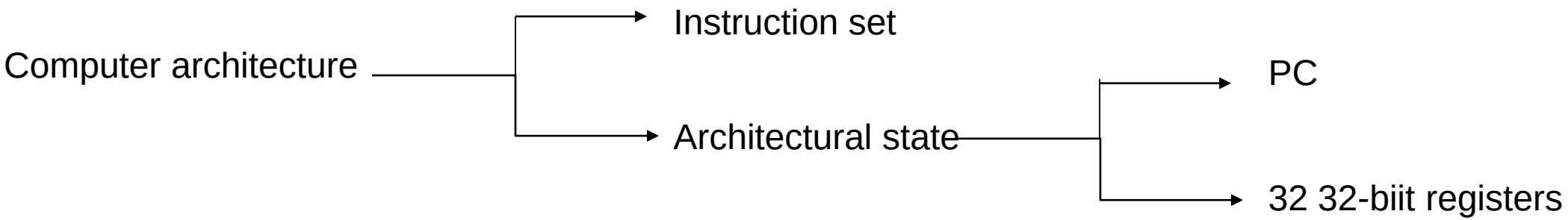
Performance Analysis.

Introduction

- We will learn how to piece together a microprocessor.
- Microarchitecture is the connection between logic and architecture.
- **Microarchitecture is the specific arrangement of registers, arithmetic logic units (ALUs), finite state machines (FSMs), memories, and other logic building blocks needed to implement an architecture**.
- A particular architecture, such as RISC-V, may have many different microarchitectures, each with different trade-offs of performance, cost, and complexity.
- They all run the same programs, but their internal designs vary widely

Architectural State and Instruction Set

- A computer architecture is defined by its instruction set and architectural state.
- The *architectural state* for the RISC-V processor consists of the program counter and the 32 32-bit registers.
- Any RISC-V microarchitecture must contain all of this state.
- Based on the current architectural state, the processor executes a particular instruction with a particular set of data to produce a new architectural state.


Computer architecture
Instruction set
Architectural state
PC
32 32-biit registers

To keep the microarchitectures easy to understand, we focus on a subset of the RISC-V instruction set.

- R-type instructions: add, sub, and, or, slt
- Memory instructions: lw, sw
- Branches: beq

Design Process

- We divide microarchitectures into two interacting parts:

➢ *data-path*

- operates on words of data.
- It contains structures such as memories, registers, ALUs, and multiplexers.
- RISC-V (RV32I) architecture use a 32-bit data-path.

➢ *control unit*

- The control unit receives the current instruction from the data-path and tells the data-path how to execute that instruction.

- control unit produces multiplexer select, register enable, and memory write signals to control the operation of the data-path.

- A good way to design a complex system is to start with hardware containing the state elements.

- These elements include **the memories and the architectural state** (the program counter and registers).

- Then, add blocks of combinational logic between the state elements to compute the new state based on the current state.

- The instruction is read from part of memory; load and store instructions then read or write data from another part of memory.

- Hence, it is often convenient to partition the overall memory into two smaller memories, one containing instructions and the other containing data.

- State elements
  - Memory
  - Architectural state
    - PC
    - Registers

State elements --- memory, PC, registers

- Memory contain
  - Instructions
  - Data
- Memory
  - Instruction memory
  - Data memory

Fig1. shows a block diagram with the four state elements: the **program counter, register file, and instruction and data memories**

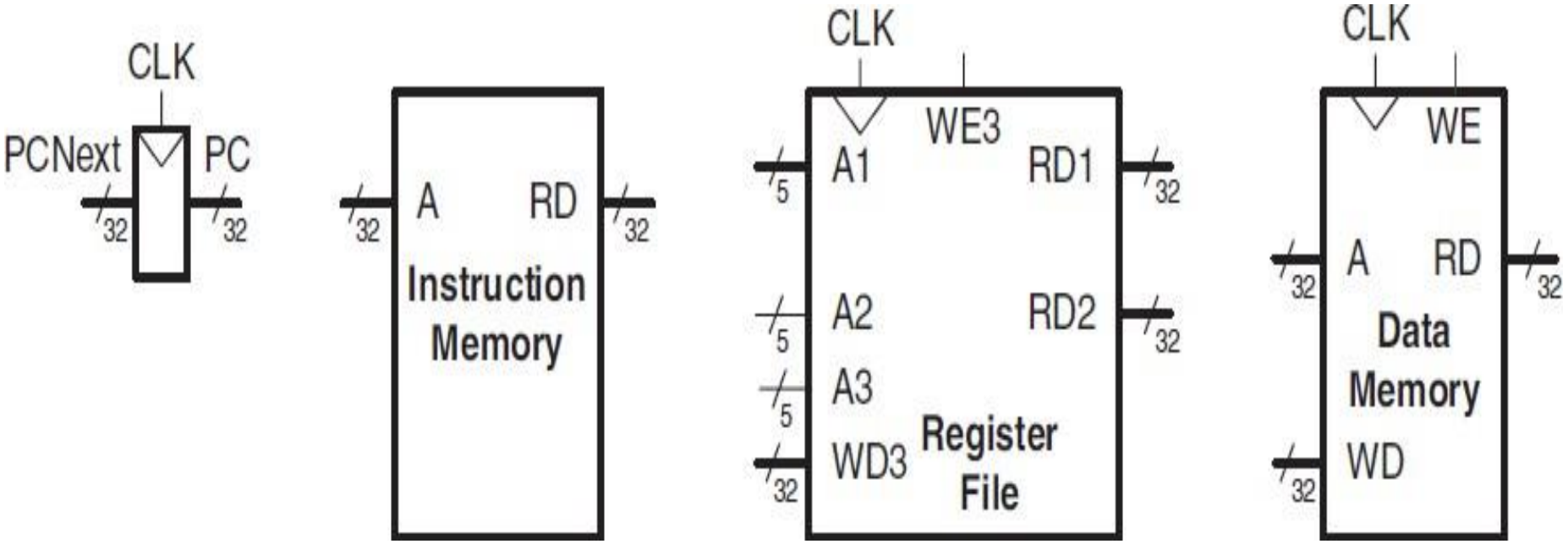


Fig1. State elements of a RISC-V processor

- The ***program counter*** (PC) points to the current instruction.
- Its input, ***PCNext***, indicates the address of the next instruction.
- The ***instruction memory*** **has a single read port.**
- It takes a 32-bit instruction address input, *A*, and reads the 32-bit data (i.e., instruction) from that address onto the read data output, *RD*.
- The 32-element × 32-bit register file holds registers x0–x31.
- The register file has **two read ports and one write port.**
- The read ports take 5-bit address inputs, *A1* and *A2*, each specifying one of 32 registers as source operands.

Port	Description
A1	5-bit address of source register 1
A2	5-bit address of source register 2
RD1	32-bit data output from register A1
RD2	32-bit data output from register A2
A3	5-bit address of destination register
WD3	32-bit data to be written
WE3	Write enable
Clock	Controls write operation

<u>Operation</u>

- Read
  - Combinational (no clock needed).
  - Values from registers at addresses A1 and A2 appear on RD1 and RD2.

- Write
  - Occurs on the rising edge of the clock.
  - If WE3 = 1, data WD3 is written to register A3
  - If A3 = x0, write is ignored

- The register file places the 32-bit register values onto read data outputs *RD1* and *RD*2.

- The write port, port 3, takes a 5-bit address input, *A3*; a 32-bit write data input, *WD3*; a write enable input, *WE3*; and a clock.

- If its write enable (*WE3*) is asserted, then the register file writes the data (*WD3*) into the specified register (*A3*) on the rising edge of the clock.

- In the figure, heavy lines indicate 32-bit data busses.
- Medium lines indicate narrower busses, such as the 5-bit address busses on the register file.
- Narrow lines indicate 1-bit wires, and
- blue lines are used for control signals, such as the register file write enable.

data memory

- Stores data used by the program.
- Used by load (lw) and store (sw) instructions
- The *data memory* has a single read/write port.
- Address input: A
- Write data input: WD
- Read data output: RD
- Write enable: WE

Operation

- Write Operation (Store Instruction)

➢ When WE = 1.

➢ On the rising edge of the clock

➢ The data WD is written into the memory location specified by address A.

$$\textbf{Mem[}A\textbf{]} \leftarrow WD$$

- Read Operation (Load Instruction)

➢ When WE = 0

➢ Memory performs a read

➢ The contents of memory at address A are placed onto the read data bus

RD← Mem[A]

- The instruction memory, register file, and data memory are all read *combinationally*.
- In other words, if the address changes, then the new data appears at *RD* after some propagation delay; no clock is involved.
- The clock controls writing only.
- These memories are written only on the rising edge of the clock.

Microarchitectures

- Three microarchitectures for the RISC-V architecture:
  - single-cycle
  - multicycle
  - Pipelined

single-cycle

- The *single-cycle microarchitecture* executes an entire instruction in one cycle.
- It is easy to explain and has a simple control unit.
- Because it completes the operation in one cycle, it does not require any nonarchitectural state.
- However, the cycle time is limited by the slowest instruction.
- Moreover, the processor requires separate instruction and data memories, which is generally unrealistic

multicycle

- The *multicycle microarchitecture* executes instructions in a series of shorter cycles.
- Multicycle microarchitecture reduces the hardware cost by reusing expensive hardware blocks, such as adders and memories.
- The multicycle processor executes only one instruction at a time, but each instruction takes multiple clock cycles.
- This processor requires only a single memory, accessing it on one cycle to fetch the instruction and on another to read or write data.
- Because they use less hardware than single-cycle processors, multicycle processors were the historical choice for inexpensive systems.

pipelined microarchitecture

- The *pipelined microarchitecture* applies pipelining to the single-cycle microarchitecture.
- It therefore can execute several instructions simultaneously, improving the throughput significantly.
- Pipelining must add logic to handle dependencies between simultaneously executing instructions.
- Pipelined processors must access instructions and data in the same cycle
- All commercial high-performance processors use pipelining today

PERFORMANCE ANALYSIS

- A particular processor architecture can have many microarchitectures with different cost and performance trade-offs.
- The cost of a processor architecture depends on the amount of hardware required and the implementation technology used.
- In general, more gates and more memory mean more cost.
- There are many ways to measure computer performance of a computer system, such as clock frequency and number of cores.
- What are the factors that a buyer consider to measure performance ?

- However, the most meaningful measure of performance is the execution time of a program.
- The computer that executes a program fastest is said to have the highest performance.
- When the actual program is not available, the total execution time of a set of similar programs is measured.
- This may be necessary if you have not written your program yet or if somebody else who does not have your program is making the measurements.
- Such collections of similar programs are called **benchmarks**.
- The execution times of benchmark programs are commonly published to provide an indication of how a processor performs.
- Dhrystone, CoreMark, and SPEC are three popular benchmarks.

- **Synthetic benchmarks**

✓ are artificial benchmark programs used to test processor performance.

✓ They are composed of important common pieces of programs

✓ They consist of commonly used operations such as loops and arithmetic instructions.

✓ Examples: Dhrystone and CoreMark.

- **Real benchmarks**

✓ are composed of actual application programs.

✓ Example: SPECspeed 2017 Integer benchmark from the Standard Performance Evaluation Corporation (SPEC) which stresses the entire system realistically.

✓ The benchmark is widely used for high-performance processors because it stresses the entire system in a representative way.

Execution Time

$$ExecutionTime = (\#instructions)\left(\frac{cycles}{instruction}\right)\left(\frac{seconds}{cycle}\right)$$

Eq, gives the execution time of a program, measured in seconds

- **Number of Instructions**

✓ Total instructions executed by the program

✓ depends on the algorithm, compiler, and ISA.

- **Cycles per Instruction (CPI)**

✓ Number of clock cycles required to execute one instruction

✓ depends on processor design and implementation.

✓ It is the reciprocal of the throughput (*instructions per cycle*, or *IPC*).

- **Seconds per Cycle**

✓ Represents the clock period, **denoted as** T_c **.**

✓ The clock period is determined by the processors's clock frequency:

$$T_c = \frac{1}{\text{Clock Frequency}}$$

- Execution time depends on **instruction count**, **CPI**, and **clock period**.
- Reducing any one of these factors reduces the total execution time.
- Therefore, **higher clock frequency alone does not guarantee better performance**.
- The challenge of the micro architect is to choose the design that minimizes the execution time while satisfying constraints on cost and/or power consumption.
- Because micro architectural decisions affect both CPI and T_c and are influenced by logic and circuit designs, determining the best choice requires careful analysis.

Q) Given a single cycle processor with a clock cycle time of 750 ps, calculate the execution time for a program containing 20 billion instructions.

Execution time (CPU time) = (no of instructions) (cycle/instruction) (seconds/cycle)

$= 20 \times 10^{9} \times 1 \times 750 \times 10^{-12}$

$= 15000 \times 10^{(9-12)}$

$= 15000 \times 10^{-3}$

$= 15$ s

SINGLE-CYCLE PROCESSOR

- Design a microarchitecture that executes instructions in a single cycle.
- begin constructing the data-path by connecting the state elements with combinational logic that can execute the various instructions.
- Control signals determine which specific instruction is performed by the data-path at any given time.
- The control unit contains combinational logic that generates the appropriate control signals based on the current instruction.
- Finally, analyze the performance of the single-cycle processor

SINGLE-CYCLE PROCESSOR: Sample Program

- Assume that we have a single cycle processor that run a short program having load, store, an R-type instruction (or) and a branch(beq)
- Suppose that the program is stored in memory starting at address 0x1000.
- The Program Counter (PC) initially contains 0x1000.
- The program demonstrates four RISC-V instruction formats: I-type, S-type, R-type, and B-type.
- Initial register and memory contents:

✓ Register **x5 = 6**

✓ Register **x9 = 0x2004**

✓ Memory location **0x2000 = 10**

- Load instruction (lw):
  - Calculates address: 0x2004 – 4 = 0x2000
  - Reads value 10 from memory
  - Stores the value in register x6
- Store instruction (sw):
  - Calculates address: 0x2004 + 8 = 0x200C
  - Writes value 10 into memory at address 0x200C
- **R-type instruction (or)**:
  - Performs bitwise OR

x4 = 6 | 10 = 0110_2 | 1010_2 = 1110_2 = 14

- **Branch instruction (beq)**:
  - Branches back to label **L7**
  - Causes the program to **repeat indefinitely**

SINGLE-CYCLE PROCESSOR: Sample Program

Address	Instruction	Type	Fields						Machine Language
			$imm_{11:0}$		rs1	f3	rd	op	
0x1000	L7: lw x6, -4(x9)	I	111111111100		01001	010	00110	0000011	FFC4A303
			$imm_{11:5}$	rs2	rs1	f3	$imm_{4:0}$	op	
0x1004	sw x6, 8(x9)	S	0000000	00110	01001	010	01000	0100011	0064A423
			funct7	rs2	rs1	f3	rd	op	
0x1008	or x4, x5, x6	R	0000000	00110	00101	110	00100	0110011	0062E233
			$imm_{12,10:5}$	rs2	rs1	f3	$imm_{4:1,11}$	op	
0x100C	beq x4, x4, L7	B	1111111	00100	00100	000	10101	1100011	FE420AE3

SINGLE-CYCLE DATAPATH

- This section explains the gradual development of the single-cycle datapath.
- New hardware components are added step by step to the state elements.
- New connections are highlighted in black (or blue for new control signals).
- Hardware already discussed is shown in gray.
- The example instruction being executed is shown at the bottom of each figure

Instruction Fetch Step

- The Program Counter (PC) holds the address of the instruction to be executed.
- The first step is to read this instruction from instruction memory. (So the first step in the data-path is instruction fetch).
- The PC output is directly connected to the address input of the instruction memory.
- The instruction memory reads (fetches) a 32-bit instruction, labeled Instr.
- Initial PC = 0x1000(Full 32-bit value is 0x00001000; leading zeros are omitted for clarity).
- Instruction memory outputs:
- Instr = lw
- Machine code: 0xFFC4A303

- **R-type instructions:**
  - `add, sub, and, or, slt`
- **I-type instruction:**
  - `lw`
- **S-type instruction:**
  - `sw`
- **B-type instructions:**
  - `beq`

R-Type

31:25	24:20	19:15	14:12	11:7	6:0
funct7	rs2	rs1	funct3	rd	op
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

I-Type

31:20	19:15	14:12	11:7	6:0
$imm_{11:0}$	rs1	funct3	rd	op
12 bits	5 bits	3 bits	5 bits	7 bits

S-Type

31:25	24:20	19:15	14:12	11:7	6:0
$imm_{11:5}$	rs2	rs1	funct3	$imm_{4:0}$	op
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

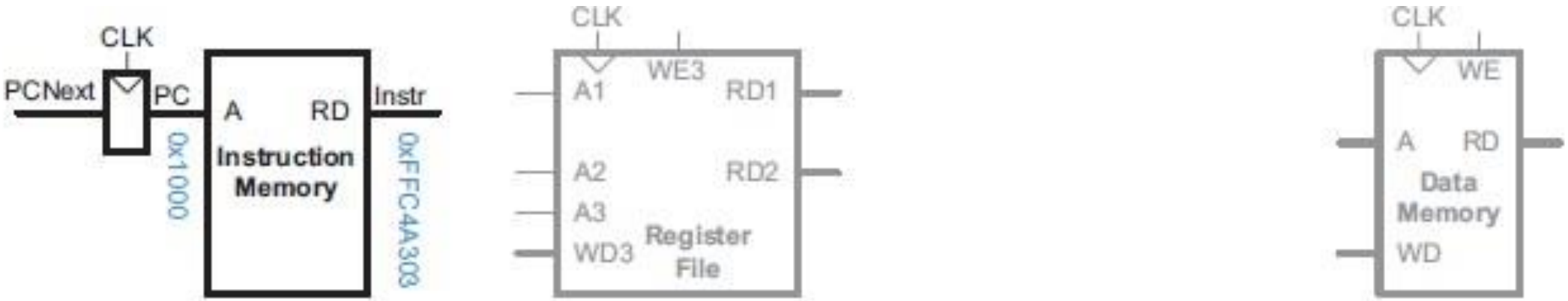
B-Type

31:25	24:20	19:15	14:12	11:7	6:0
$imm_{12,10:5}$	rs2	rs1	funct3	$imm_{4:1,11}$	op
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

SINGLE-CYCLE PROCESSOR: SINGLE-CYCLE DATAPATH: LW FETCH

lw : load word instruction

STEP 1: Fetch instruction

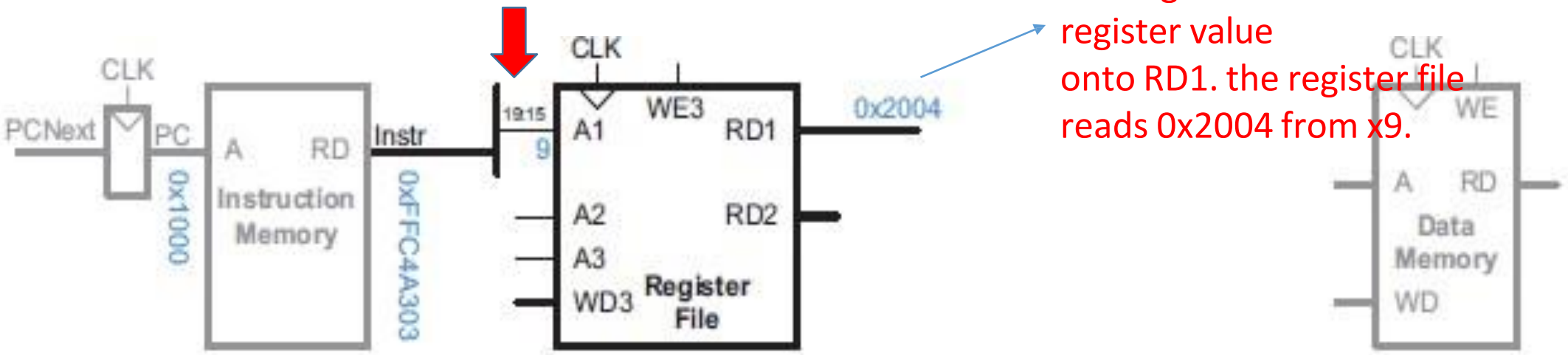


Address	Instruction	Type	Fields				Machine Language
			$imm_{11:0}$	rs1	f3	rd	op	
0x1000	L7: lw x6, -4(x9)	I	111111111100	01001	010	00110	0000011	FFC4A303

Figure 7.3 Fetch instruction from memory

SINGLE-CYCLE PROCESSOR: SINGLE-CYCLE DATA-PATH LW REG READ

STEP 2: Read source operand (rs1) from RF



The register file reads the register value onto RD1. the register file reads 0x2004 from x9.

Address	Instruction	Type	Fields					Machine Language
			$imm_{11:0}$	rs1	f3	rd	op	
0x1000	L7: lw x6, -4(x9)	I	111111111100	01001	010	00110	0000011	FFC4A303

Figure 7.4 Read source operand from register file

*Instr*19:15.
These bits of the instruction connect to the *A1* address input of the register file

Single-Cycle Datapath: lw Immediate

- The lw instruction also requires an offset. The offset is stored in the 12-bit immediate field of the instruction, *Instr*31:20.
- It is a signed value, so it must be sign-extended to 32 bits. Sign extension simply means copying the sign bit into the most significant bits: *ImmExt*31:12 = *Instr*31 , and *ImmExt*11:0 = *Instr*31:20.
- Sign extension is performed by an Extend unit.
- which receives the 12-bit signed immediate in *Instr*31:20 and produces the 32-bit sign-extended immediate, *ImmExt*.
- In our example, the two's complement immediate −4 is 12-bit immediate -4 (0xFFC) and sign-extends it to 32 bits: 0xFFFFFFFC.

STEP 3: Extend the immediate

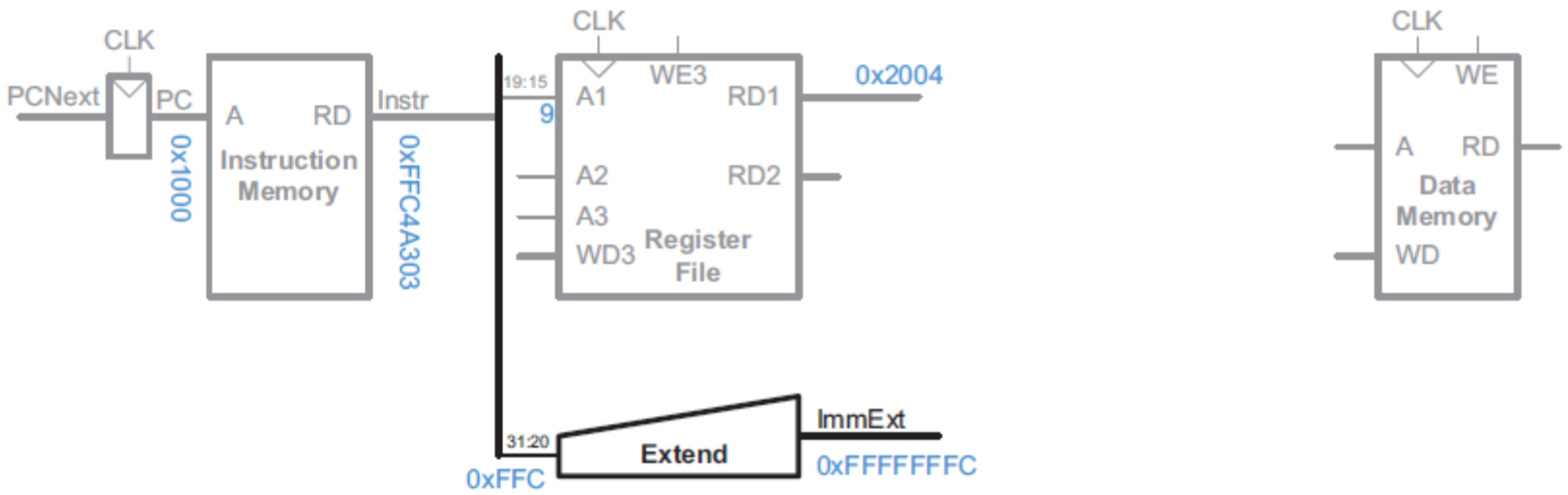
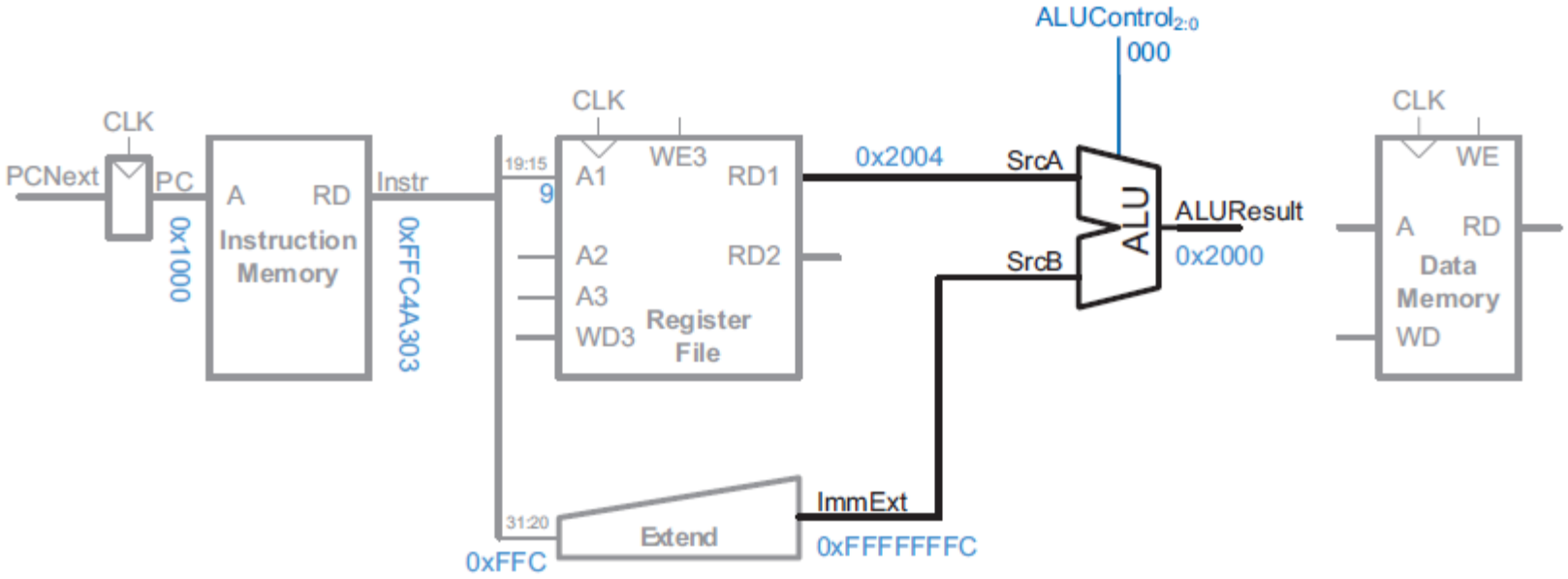


Figure 7.5 Sign-extend the immediate

Single-Cycle Datapath: lw Address

- The processor adds the base address to the offset to find the address to read from memory.
- introduces an ALU to perform this addition.
- The ALU receives two operands, *SrcA* and *SrcB*. *SrcA* is the base address from the register file, and *SrcB* is the offset from the sign-extended immediate, *ImmExt*.
- In our example, the ALU computes 0x2004 + 0xFFFFFFFC = 0x2000.
- Again, this is a 32-bit value, but we omit the leading zeros

STEP 4: Compute the memory address



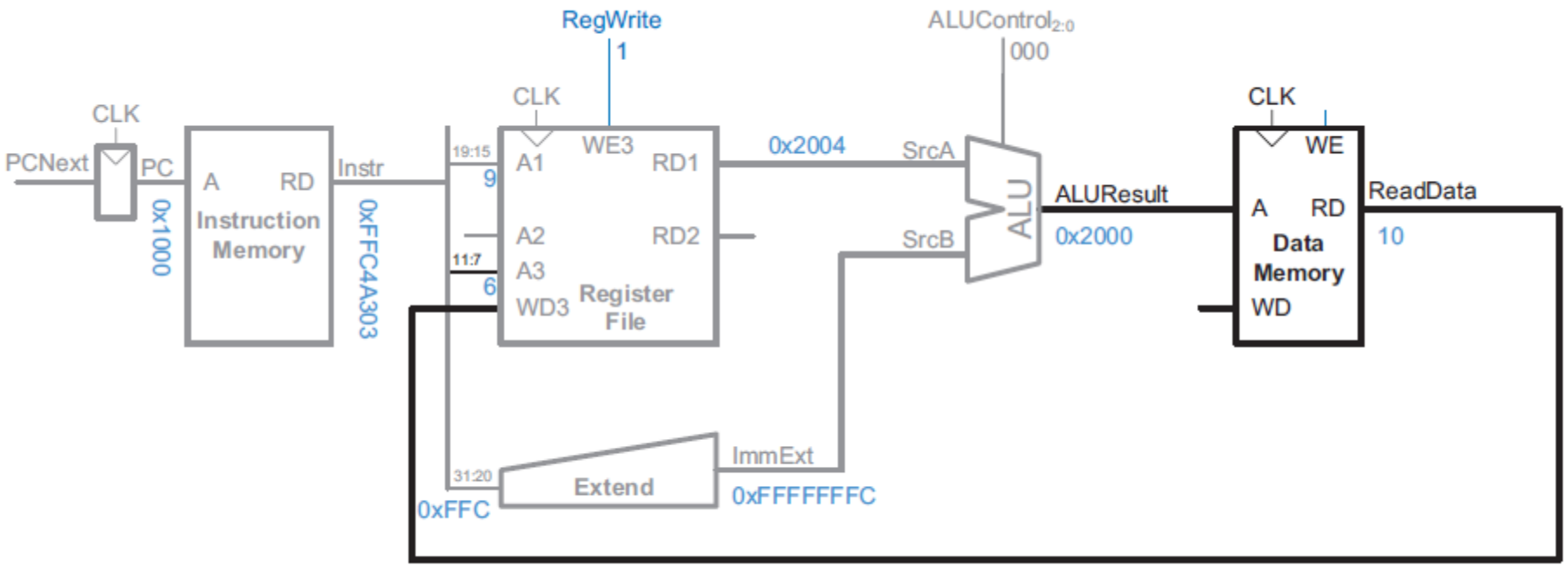
Address	Instruction	Type	Fields					Machine Language
			$imm_{11:0}$	rs1	f3	rd	op	
0x1000	L7: lw x6, -4(x9)	I	111111111100	01001	010	00110	0000011	FFC4A303

Figure 7.6 Compute memory address

Single-Cycle Datapath: LDR Mem Read

- This memory address from the ALU is provided to the address (*A*) port of the data memory.
- The data is read from the data memory onto the *ReadData* bus and then written back to the destination register at the end of the cycle.
- Port 3 of the register file is the write port.
- lw's destination register, indicated by the **rd** field (*Instr*11:7), is connected to *A3*, port 3's address input.
- The *ReadData* bus is connected to *WD3*, port 3's write data input.
- A control signal called *RegWrite* (register write) is connected to *WE3*, port 3's write enable input, and is asserted during the lw instruction so that the data value is written into the register file.
- In our example, the processor reads 10 from address 0x2000 in the data memory and puts that value (10) into x6 in the register file.

STEP 5: Read data from memory and write it back to register file



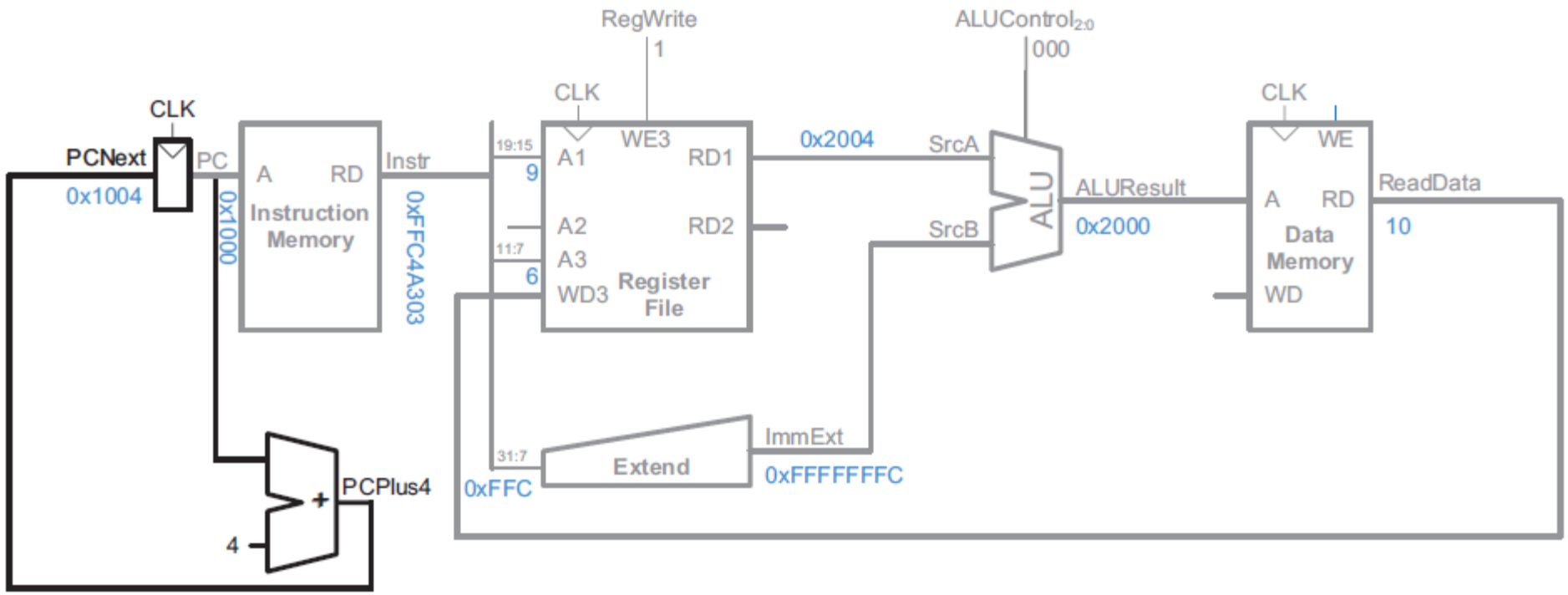
Address	Instruction	Type	Fields					Machine Language
			$imm_{11:0}$	rs1	f3	rd	op	
0x1000	L7: lw x6, -4(x9)	I	111111111100	01001	010	00110	0000011	FFC4A303

Figure 7.7 Read memory and write result back to register file

Single-Cycle Datapath: PC Increment

- While the instruction is being executed, the processor must also compute the address of the next instruction, *PCNext*.
- Because instructions are 32 bits (4 bytes), the next instruction is at PC+4.
- In our example, *PCNext* = 0x1000 + 4 = 0x1004.
- The new address is written into the program counter on the next rising edge of the clock.
- This completes the datapath for the lw instruction.

STEP 6: Determine address of next instruction



Address	Instruction	Type	Fields					Machine Language
			$imm_{11:0}$	rs1	f3	rd	op	
0x1000	L7: lw x6, -4(x9)	I	111111111100	01001	010	00110	0000011	FFC4A303

Figure 7.8 Increment program counter

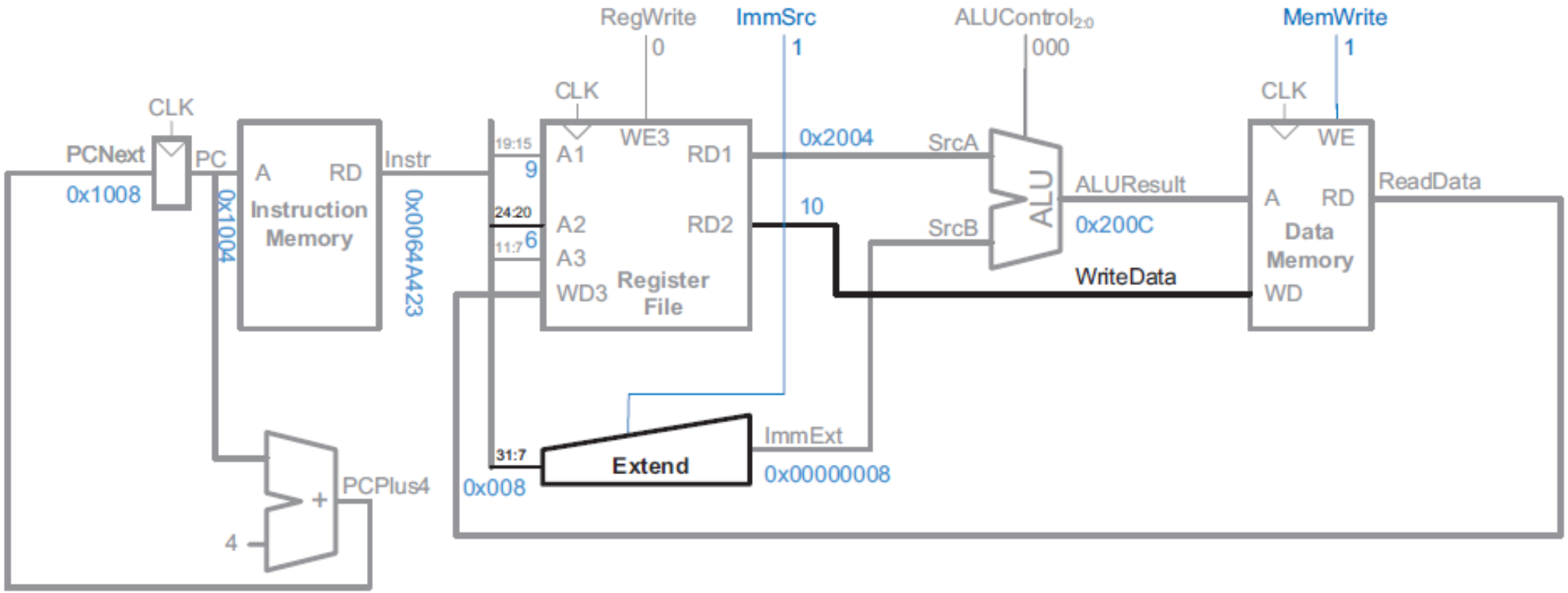
Single-Cycle Datapath: sw

S-Type

31:25	24:20	19:15	14:12	11:7	6:0
$imm_{11:5}$	rs2	rs1	funct3	$imm_{4:0}$	op
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

- sw, which is an S-type instruction.
- Like lw, sw reads a base address from port 1 of the register file and sign-extends the immediate.
- The ALU adds the base address to the immediate to find the memory address.
- All of these functions are already supported in the datapath, but the 12-bit signed immediate is stored in Instr31:25,11:7 (instead of Instr31:20, as it was for lw).
- Thus, the Extend unit must be modified to also receive these additional bits, Instr11:7.

- The sw instruction also reads a second register from the register file and writes its contents to the data memory.
- The register is specified in the rs2 field, **Instr24:20**, which is connected to the address **2 (A2)** input of the register file.
- The register's contents are read onto the read data **2 (RD2) output**, which, in turn, is connected to the **write data (WD)** input of the data memory.
- The write enable port of the data memory, WE, is controlled by MemWrite.
- For an sw instruction: *MemWrite* = 1 to write the data to memory;
- *ALUControl* = 000 to add the base address and offset; and *RegWrite* = 0, because nothing should be written to the register file.
- Note that data is still read from the address given to the data memory, but this *ReadData* is ignored because *RegWrite* = 0.



Address	Instruction	Type	Fields						Machine Language
			$imm_{11:5}$	rs2	rs1	f3	$imm_{4:0}$	op	
0x1004	sw x6, 8(x9)	S	0000000	00110	01001	010	01000	0100011	0064A423

Figure 7.9 Write data to memory for sw instruction

- In our example, the PC is 0x1004.
- Thus, the instruction memory reads out the sw instruction, 0x0064A423.
- The register file reads 0x2004 (the base address) from x9 and 10 from x6 while the Extend unit extends the immediate offset 8 from 12 to 32 bits.
- The ALU computes 0x2004 + 8 = 0x200C.
- The data memory writes 10 to address 0x200C.
- Meanwhile, the PC is incremented to 0x1008.

Single-Cycle Datapath: R-Type

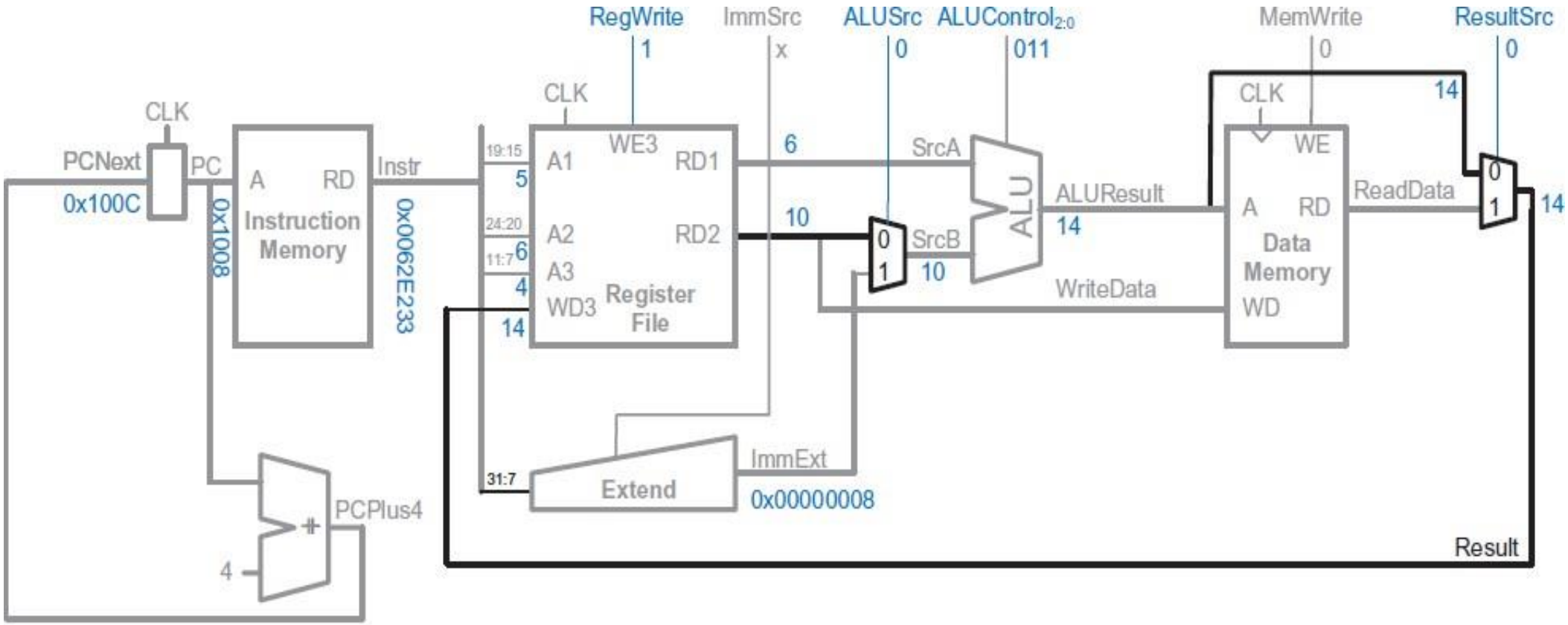
- **Instructions:** add, sub, and, or, slt.
- All of these instructions read two source registers from the register file, perform some ALU operation on them, and write the result back to the destination register.
- They differ only in the specific ALU operation.
- Hence, they can all be handled with the same hardware but with different *ALUControl* signals.

R-Type

31:25	24:20	19:15	14:12	11:7	6:0
funct7	rs2	rs1	funct3	rd	op
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

Single-Cycle Datapath: R-Type

- The datapath reads **rs1** and **rs2** from ports 1 and 2 of the register file and performs an ALU operation on them.
- Write *ALUResult* to **rd**
- We introduce **a multiplexer and a new select signal**, ***ALUSrc*, to select between *ImmExt* and *RD2*** as the second ALU source, *SrcB*.
- For lw and sw, *ALUSrc* is 1 to select *ImmExt*;
- for R-type instructions, ***ALUSrc* is 0** to select **the register file output** *RD2* as *SrcB*.



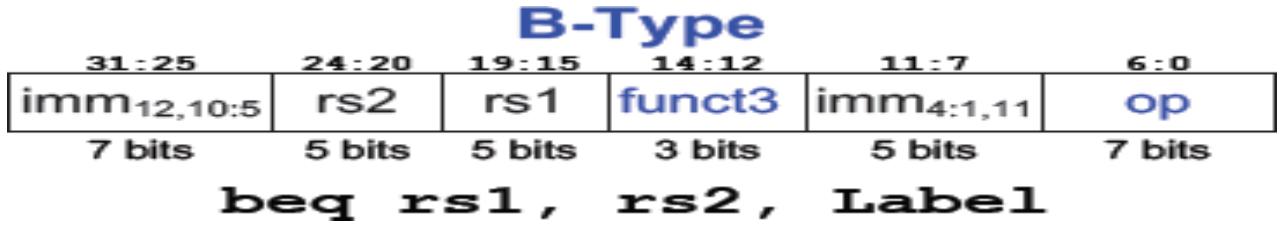
Address	Instruction	Type	Fields						Machine Language
			funct7	rs2	rs1	f3	rd	op	
0x1008	or x4, x5, x6	R	0000000	00110	00101	110	00100	0110011	0062E233

Figure 7.10 Datapath enhancements for R-type instructions

- We add the Result multiplexer to choose the proper *Result* based on the type of instruction.
- The multiplexer select signal *ResultSrc* is 0 for R-type instructions to choose *ALUResult* as *Result*; *ResultSrc* is 1 for lw to choose *ReadData*.
- We do not care about the value of *ResultSrc* for sw because it does not write the register file.
- In our example, the PC is 0x1008.
- Thus, the instruction memory reads out the or instruction 0x0062E233.
- The register file reads source operands 6 from x5 and 10 from x6.
- *ALUControl* is 011, so the ALU computes 6 | 10 = 0110_2 | 1010_2 = 11102 = 14.
- The result is written back to x4.
- Meanwhile, the PC is incremented to 0x100C.

Single-Cycle Datapath: beq

- **Calculate branch target address:** PCTarget = PC + imm.

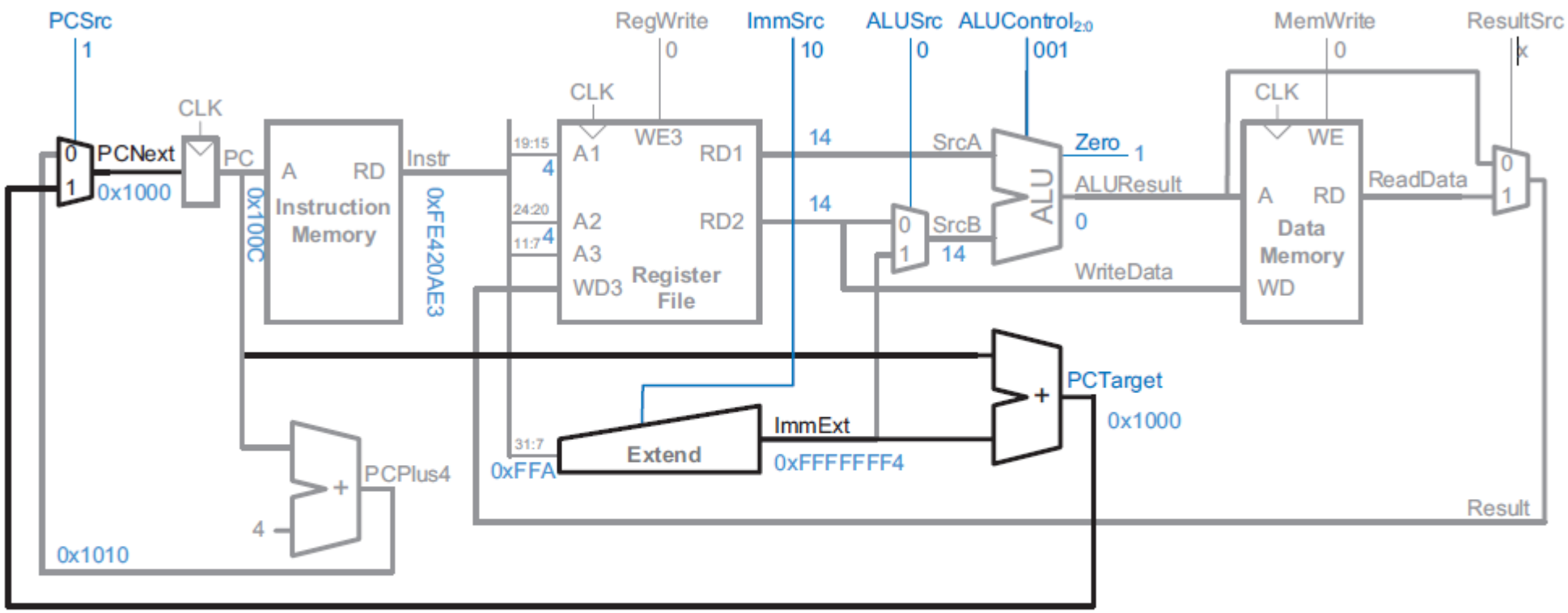


- beq compares two registers.
- If they are equal, it takes the branch by adding the branch offset to the program counter (PC).
- The branch offset is a 13-bit signed immediate stored in the 12-bit immediate field of the B-type instruction.
- *ImmSrc* is increased to 2 bits, using the encoding from Table 7.1.
- *ImmExt* is now either the sign-extended immediate (when *ImmSrc* = 00 or 01) or the branch offset (when *ImmSrc* = 10).

- The two source registers are compared by computing **(*SrcA – SrcB*)** using the ALU.
- If *ALUResult* is 0, as indicated by the ALU's *Zero* flag, the registers are equal.
- We add a multiplexer to choose *PCNext* from either *PCPlus4* or *PCTarget*.
- *PCTarget* is selected if the instruction is a branch and the *Zero* flag is asserted.
- For beq, *ALUControl* = 001, so that the ALU performs a subtraction.
- *ALUSrc* = 0 to choose *SrcB* from the register file.
- *RegWrite* and *MemWrite* are 0, because a branch does not write to the register file or memory.

Table 7.1 ***ImmSrc*** **encoding**

ImmSrc	ImmExt	Type	Description
00	{{20{*Instr*[31]}}, *Instr*[31:20]}	I	12-bit signed immediate
01	{{20{*Instr*[31]}}, *Instr*[31:25], *Instr*[11:7]}	S	12-bit signed immediate
10	{{20{*Instr*[31]}}, *Instr*[7], *Instr*[30:25], *Instr*[11:8], 1'b0}	B	13-bit signed immediate



Address	Instruction	Type	Fields						Machine Language
			$imm_{12,10:5}$	rs2	rs1	f3	$imm_{4:1,11}$	op	
0x100C	beq x4, x4, L7	B	1111111	00100	00100	000	10101	1100011	FE420AE3

Figure 7.11 Datapath enhancements for beq

- In our example, the PC is 0x100C, so the instruction memory reads out the beq instruction 0xFE420AE3.
- Both source registers are x4, so the register file reads 14 on both ports.
- The ALU computes 14 – 14 = 0, and the ***Zero* flag is asserted.** Meanwhile, the Extend unit produces 0xFFFFFFF4 (i.e., −12), which is added to *PC* to obtain *PCTarget* = 0x1000.
- The PCNext mux chooses *PCTarget* as the next PC and branches back to the start of the code at the next clock edge.

Single-Cycle RISC-V Processor

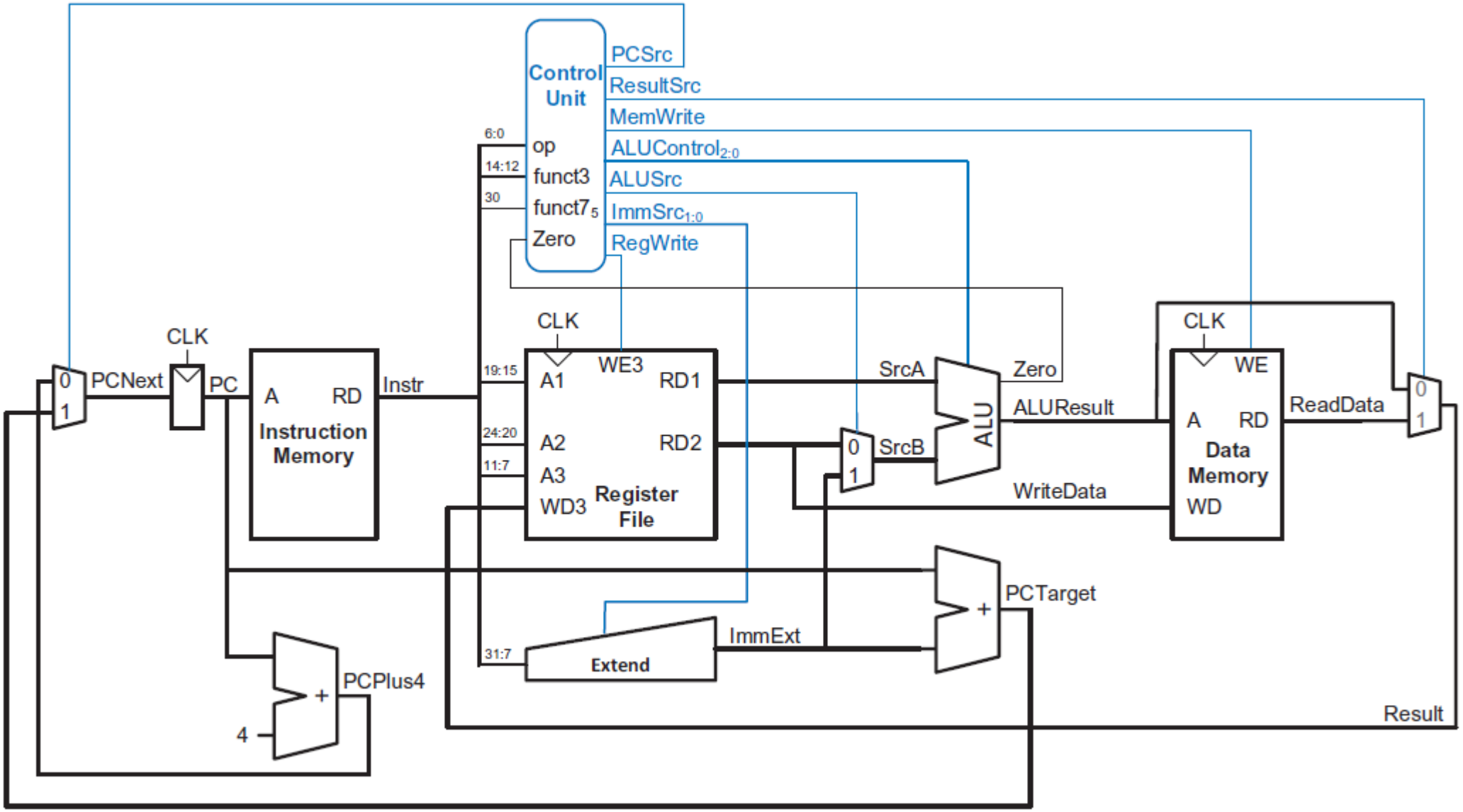


Figure 7.12 Complete single-cycle processor

Single-Cycle Control

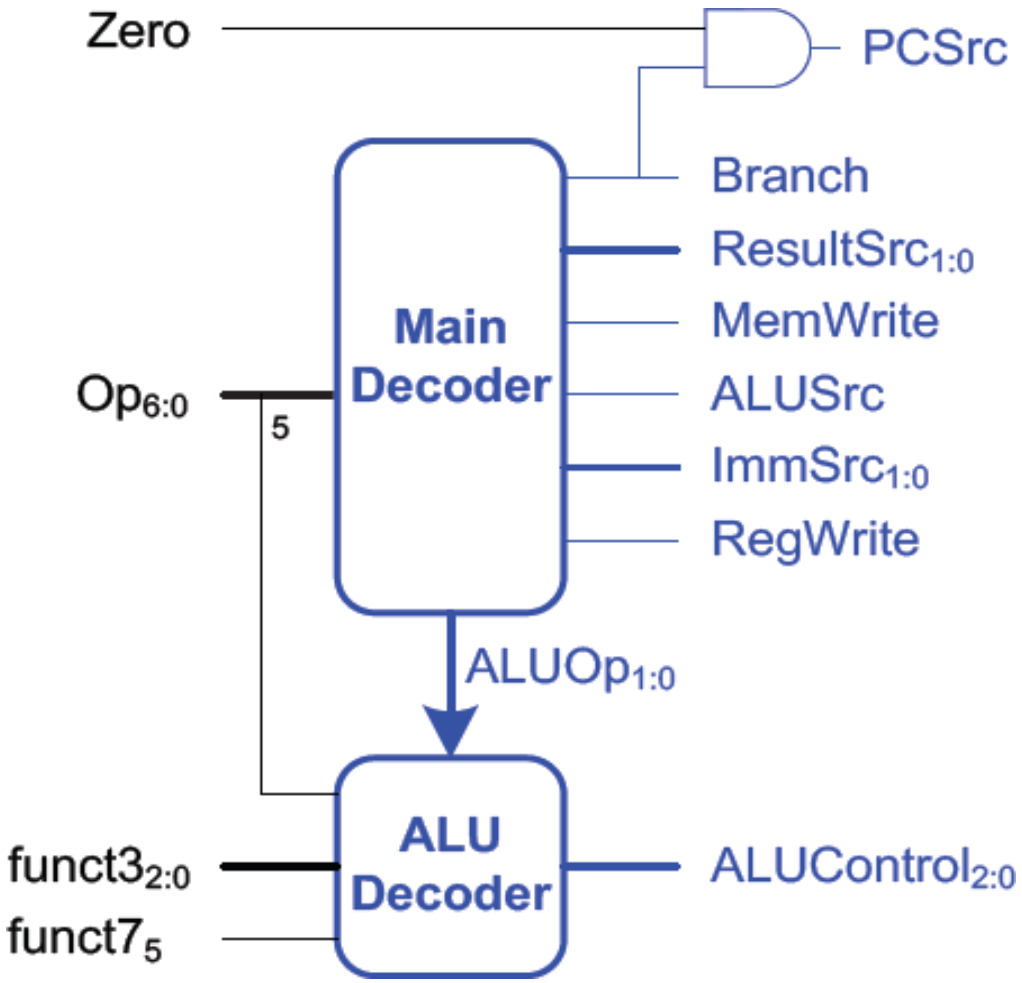


Figure 7.13 Single-cycle processor control unit

Single-Cycle Control

- The single-cycle processor's control unit computes the control signals based on **op**, **funct3**, and **funct7**.
- the control unit is also referred to as the *controller* or the *decoder*, because it decodes what the instruction should do.
- We partition it into two major parts:
  - the **Main Decoder**, which produces most of the control signals, &
  - the **ALU Decoder**, which determines what operation the ALU performs.

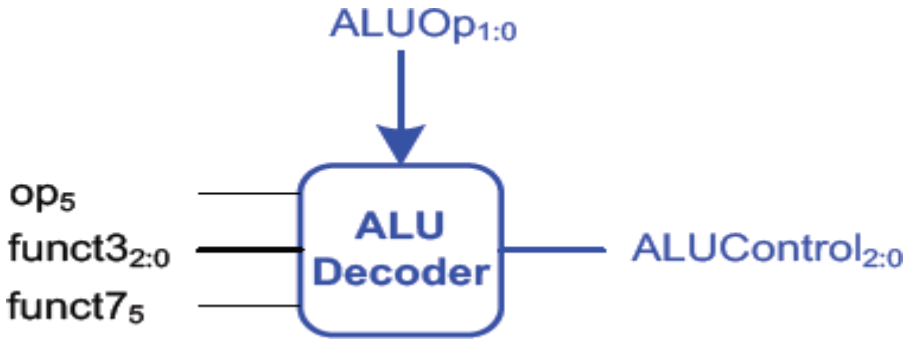
Instruction	Op	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp
lw	0000011	1	00	1	0	1	0	00
sw	0100011	0	01	1	1	x	0	00
R-type	0110011	1	xx	0	0	0	0	10
beq	1100011	0	10	0	0	x	1	01

- Table shows the control signals that the Main Decoder produces.
- The Main Decoder determines the instruction type from the opcode and then produces the appropriate control signals for the datapath.
- The Main Decoder generates most of the control signals for the datapath.
- It also produces internal signals *Branch* and *ALUOp*, signals used within the controller.

- The ALU Decoder produces *ALUControl* based on *ALUOp* and **funct3**.
- In the case of the sub and add instructions, the ALU Decoder also uses **funct7$_5$** and **op$_5$** to determine *ALUControl* as given in in Table.

ALUOp	funct3	{op$_5$, funct7$_5$}	ALUControl	Instruction
00	x	x	000 (add)	`lw, sw`
01	x	x	001 (subtract)	`beq`
10	000	00, 01, 10	000 (add)	`add`
	000	11	001 (subtract)	`sub`
	010	x	101 (set less than)	`slt`
	110	x	011 (or)	`or`
	111	x	010 (and)	`and`

- *ALUOp* of 00 indicates add (e.g., to find the address for loads or stores).
- *ALUOp* of 01 indicates subtract (e.g., to compare two numbers for branches).
- *ALUOp* of 10 indicates an R-type ALU instruction where the ALU Decoder must look at the **funct3** field (and sometimes also the $\mathbf{op_5}$ and $\mathbf{funct7_5}$ bits) to determine which ALU operation to perform (e.g., add, sub, and, or, slt).



Example: SINGLE-CYCLE PROCESSOR OPERATION

Q) Determine the values of the control signals and the portions of the datapath that are used when executing an "**and**" instruction.

Solution:

- Figure 7.14 illustrates the control signals and flow of data during execution of an and instruction. The PC points to the memory location holding the instruction; the instruction memory outputs this instruction.
- The main flow of data through the register file and ALU is represented with a heavy blue line.
- The register file reads the two source operands specified by *Instr*.
- *SrcB* should come from the second port of the register file (not *ImmExt*), so *ALUSrc* must be 0.
- The ALU performs a bitwise AND operation, so *ALUControl* must be 010.
- The result comes from the ALU, so *ResultSrc* is 0, and the result is written to the register file, so *RegWrite* is 1.
- The instruction does not write memory, so *MemWrite* is 0.

- The updating of *PC* with *PCPlus4* is shown by a heavy gray line.
- *PCSrc* is 0 to select the incremented PC.

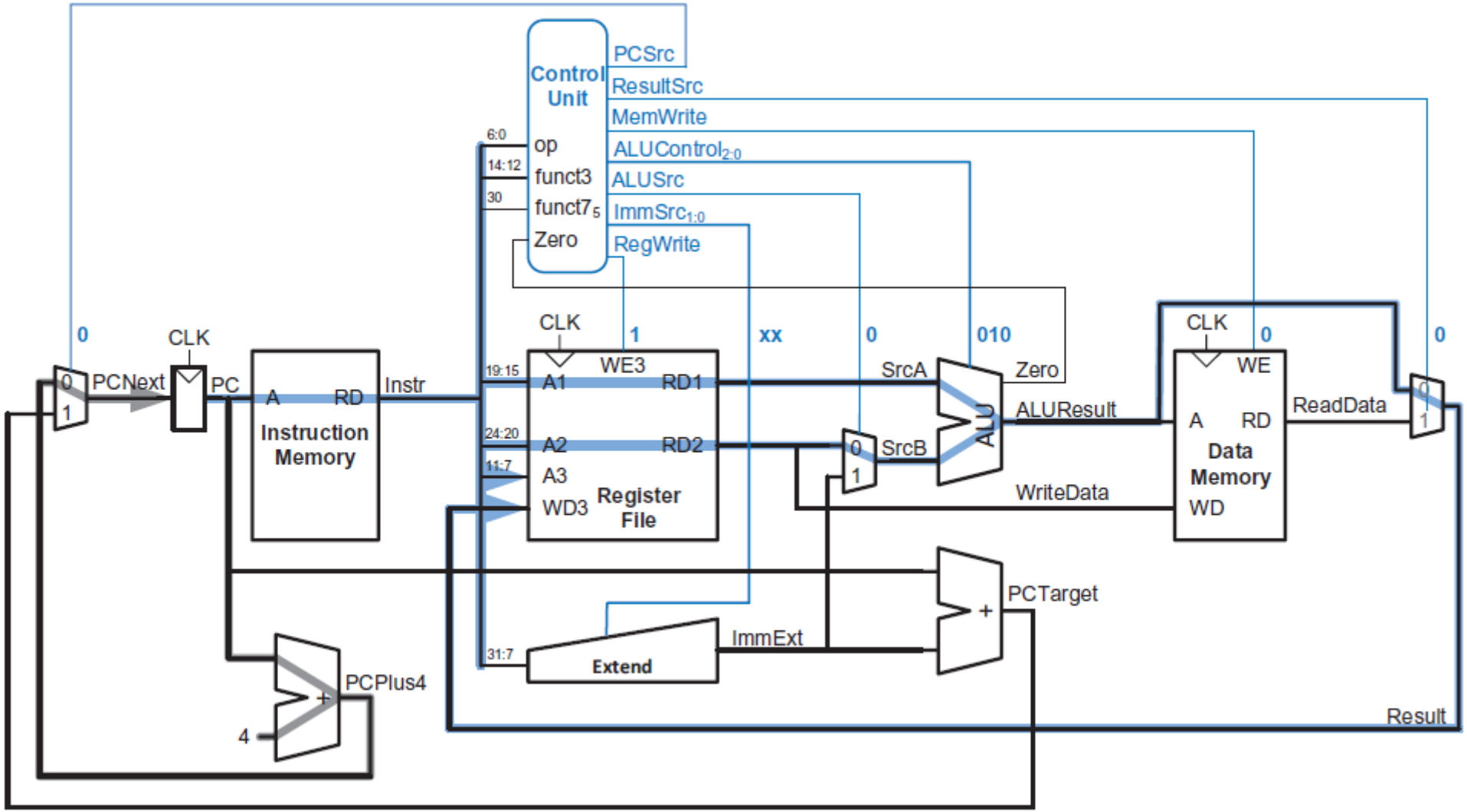


Figure 7.14 Control signals and data flow while executing an and instruction

Q) Determine the values of the control signals and the portions of the datapath that are used when executing an "**jal**" instruction.

Solution:

- Fig. show how to change the RISC-V single-cycle processor to support the jump and link (jal) instruction.
- **jal** writes PC+4 to rd and changes PC to the jump target address, PC + imm**.**
- The processor calculates the jump target address, the value of PCNext, by adding PC to the 21-bit signed immediate encoded in the instruction.
- The least significant bit of the immediate is always 0 and the next 20 most significant bits come from Instr31:12.
- This 21-bit immediate is then sign-extended.
- The datapath already has hardware for adding PC to a sign-extended immediate, selecting this as the next PC, computing PC+4, and writing a value to the register file.
- Hence, in the datapath, we must only modify the Extend unit to sign-extend the 21-bit immediate and expand the Result multiplexer to choose PC+4 (i.e., PCPlus4)

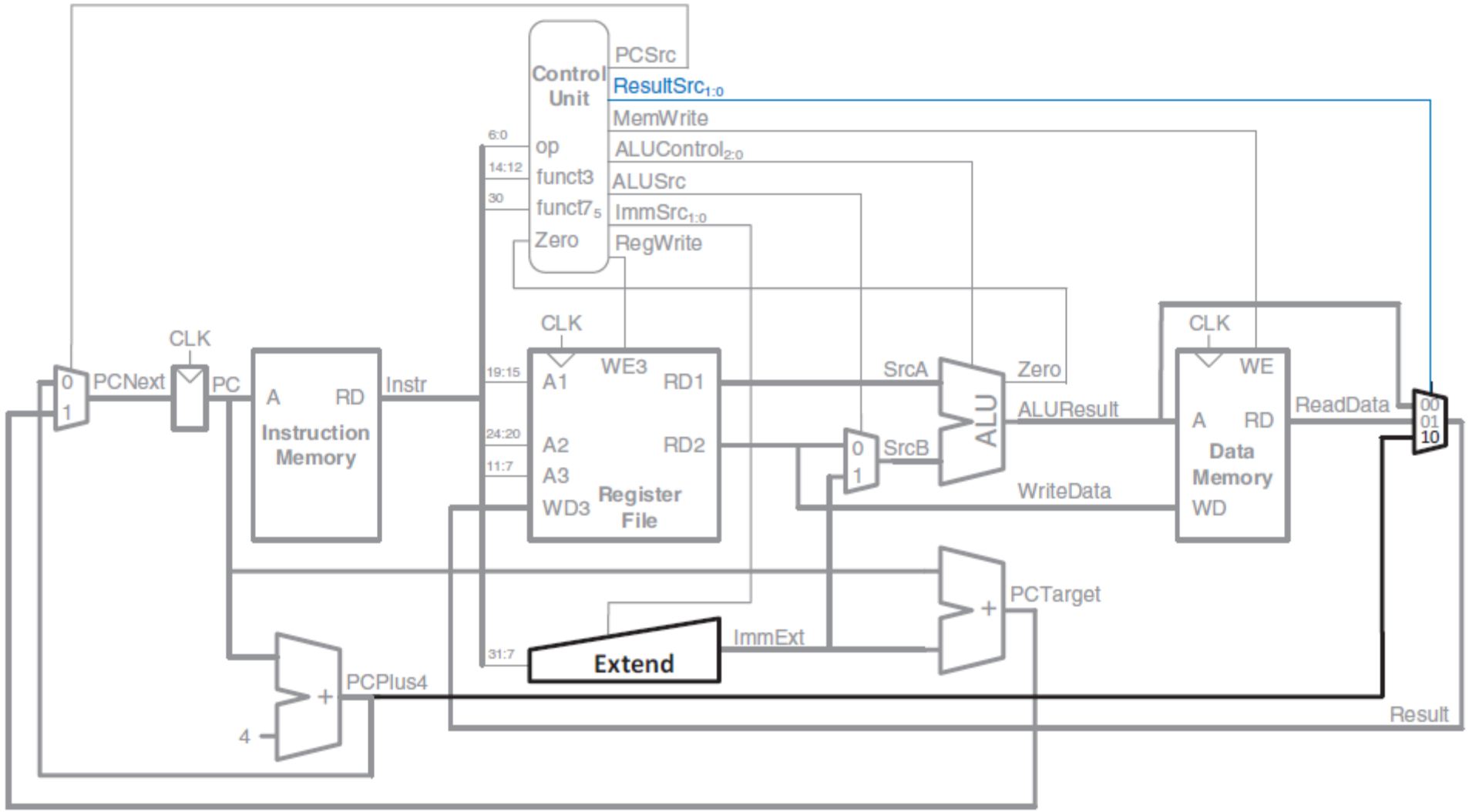


Figure 7.15 Enhanced datapath for `jal`

- The control unit needs to set *PCSrc* = 1 for the jump.
- To do this, we add an OR gate and another control signal, *Jump*, as shown in Figure 7.16.
- When *Jump* asserts, *PCSrc* = 1 and *PCTarget* (the jump target address) is selected as the next PC.

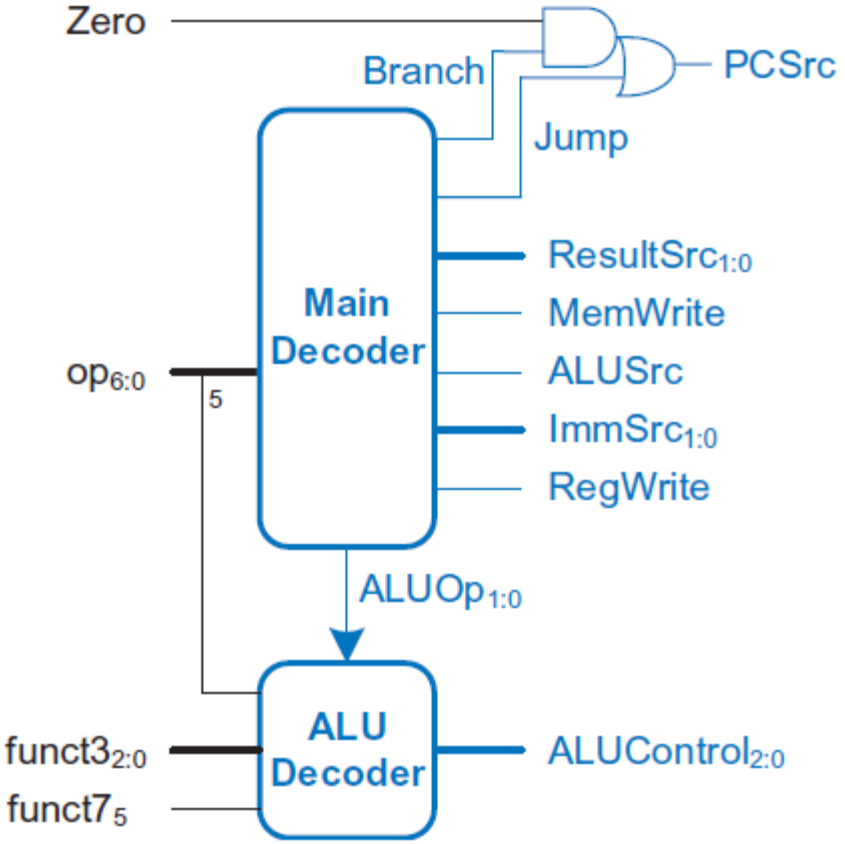


Figure 7.16 Enhanced control unit for `jal`

Single-Cycle Performance

- The execution time of a program is the product of the number of instructions, the cycles per instruction, and the cycle time.
- Each instruction in the single-cycle processor takes one clock cycle, so the clock cycles per instruction (CPI) is 1.
- The cycle time is set by the critical path.
- In our processor, the **lw** instruction is the most time consuming and involves the critical path.

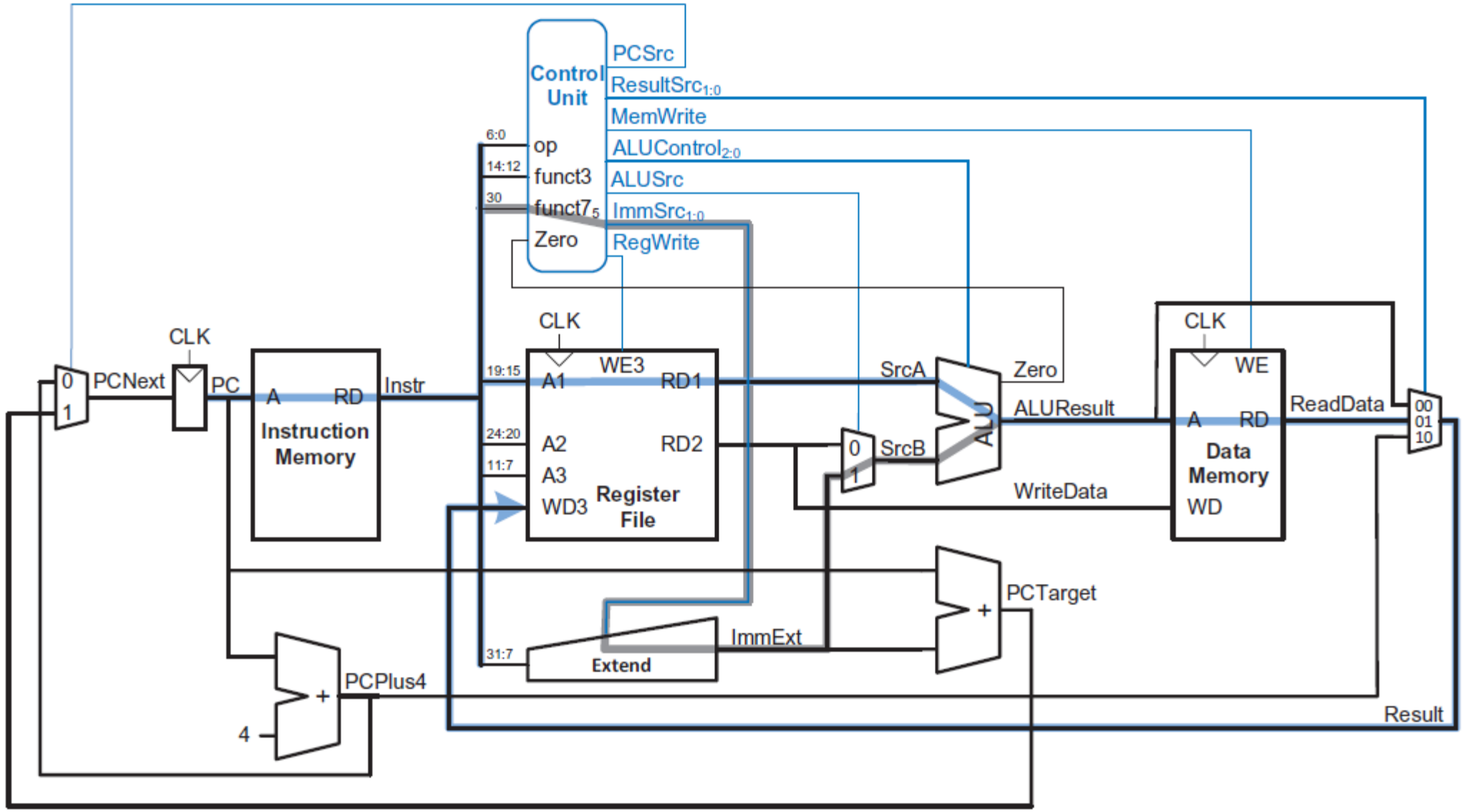


Figure 7.17 Critical path for `lw`

- As indicated by heavy blue lines, the critical path starts with the PC loading a new address on the rising edge of the clock.
- The instruction memory then reads the new instruction, and the register file reads **rs1** as *SrcA*.
- While the register file is reading, the immediate field is sign-extended based on *ImmSrc* and selected at the SrcB multiplexer (path highlighted in gray).
- The ALU adds *SrcA* and *SrcB* to find the memory address.
- The data memory reads from this address, and the Result multiplexer selects *ReadData* as *Result*.
- Finally, *Result* must set up at the register file before the next rising clock edge so that it can be properly written.

- Hence, the cycle time of the single-cycle processor is

$$T_{c_single} = t_{pcq_PC} + t_{mem} + \max[t_{RFread}, \quad t_{dec} + t_{ext} + t_{mux}] + t_{ALU} + t_{mem} + t_{mux} + t_{RFsetup}$$

- In most implementation technologies, the ALU, memory, and register file are substantially slower than other combinational blocks. Therefore, the critical path is through the register file—not through the decoder (controller), Extend unit, and multiplexer.

$$T_{c_single} = t_{pcq_PC} + 2t_{mem} + t_{RFread} + t_{ALU} + t_{mux} + t_{RFsetup}$$

Single Cycle Processor performance example

Ben Bitdiddle is contemplating building the single-cycle processor in a 7-nm CMOS manufacturing process. He has determined that the logic elements have the delays given in Table 7.7. Help him compute the execution time for a program with 100 billion instructions.

Element	Parameter	Delay (ps)
Register clk-to-Q	t_{pcq}	40
Register setup	t_{setup}	50
Multiplexer	t_{mux}	30
AND-OR gate	$t_{AND\text{-}OR}$	20
ALU	t_{ALU}	120
Decoder (control unit)	t_{dec}	25
Extend unit	t_{ext}	35
Memory read	t_{mem}	200
Register file read	t_{RFread}	100
Register file setup	$t_{RFsetup}$	60

Table 7.7 Delay of circuit elements

$$ExecutionTime = (\#instructions)\left(\frac{cycles}{instruction}\right)\left(\frac{seconds}{cycle}\right)$$ Eq no 1

$$T_{c_single} = t_{pcq_PC} + 2t_{mem} + t_{RFread} + t_{ALU} + t_{mux} + t_{RFsetup}$$ Eq no 2

Solution:

According to Equation 2, the cycle time of the single-cycle processor is T_{c_single} = 40 + 2(200) + 100 + 120 + 30 + 60 = 750 ps. According to Equation 7.1, the total execution time is

T_{single} = (100 × 10^9 instruction) (1 cycle/instruction) (750 × 10^{-12} s/cycle)

= 75 seconds

Element	Parameter	Delay (ps)
Register clock-to-Q	t_{pcq_PC}	40
Register setup	t_{setup}	50
Multiplexer	t_{mux}	25
ALU	t_{ALU}	120
Decoder (Control Unit)	t_{dec}	70
Memory read	t_{mem}	200
Register file read	t_{RFread}	100
Register file setup	$t_{RFsetup}$	60

Single-Cycle Performance Example

Element	Parameter	Delay (ps)
Register clock-to-Q	t_{pcq_PC}	40
Register setup	t_{setup}	50
Multiplexer	t_{mux}	25
ALU	t_{ALU}	120
Decoder (Control Unit)	t_{dec}	70
Memory read	t_{mem}	200
Register file read	t_{RFread}	100
Register file setup	$t_{RFsetup}$	60

$$
\begin{aligned}
T_{c1} &= t_{pcq_PC} + 2t_{mem} + t_{dec} + t_{RFread} + t_{ALU} + 2t_{mux} + t_{RFsetup} \\
&= [50 + 2(200) + 70 + 100 + 120 + 2(25) + 60]\ \text{ps} \\
&= \mathbf{840\ ps}
\end{aligned}
$$

Single-Cycle Performance Example

Program with 100 billion instructions:

Execution Time = # instructions x CPI x T_C

$= (100 \times 10^9)(1)(840 \times 10^{-12}\ s)$

= **84 seconds**